

Laboratorio 4 — AMQP + tercer protocolo (HTTPS) frente a MQTT hacia Azure IoT Central

Curso: IoT + Cloud + Sistemas Distribuidos — UNAB | **Autores:** Juan Felipe Caro Niño, Santiago Cabeza Méndez | **Fecha:** 6 de octubre de 2026

App IoT Central: UNAB-Ambiental — Monitoreo de Espacios Académicos (`laboratorio1caro.azureiotcentral.com`), plantilla «Nodo Ambiental de Aula (ESP32)»; 3 dispositivos nuevos: `lab4-amqp`, `lab4-https`, `lab4-mqtt` (los dispositivos del Lab 3 no se tocaron).

Repositorio: github.com/jcaro819/laboratorio4-amqp-https-iot-central (scripts, README sin secretos, /evidencias, /mediciones con CSV y capturas .pcap)

1. Implementación y entorno

Variables (las mismas del Lab 3, en los tres protocolos): `temperature`, `humidity`, `illuminance` — JSON de ~56 B, valores aleatorios en rangos realistas. Todos aprovisionan por DPS (HTTPS + SAS, ~2,9 s en las 4 corridas, igual para todos y por eso excluido de la comparación) y reciben el mismo hub asignado.

- AMQP 1.0 (`amqp_proton.py`):** el SDK `azure-iot-device` de Python (v2.14.0) no habla AMQP — verificado: 0 referencias a «amqp» en su código; solo MQTT (y MQTT/WebSockets) y HTTP para DPS/blobs. Se usó un cliente AMQP 1.0 explícito con `python-qpidd-proton`: TLS 1.2 al `hub:5671`, SASL PLAIN con usuario `<id>@sas.<hub>` y el token SAS del dispositivo como contraseña, enlace emisor a `/devices/<id>/messages/events`; el cuerpo se envía como sección *Data* con `content_type=application/json`. Cada mensaje se confirma con una disposición `accepted`.
- MQTT (línea base, `mqtt_baseline.py`):** `paho-mqtt` 3.1.1 sobre TLS 8883, QoS 1 (confirmación = PUBACK), mismo patrón del Lab 3 pero medido en el mismo entorno que los otros dos.
- Tercer protocolo — HTTPS (`https_client.py`):** POST `https://<hub>/devices/<id>/messages/events` con el SAS en *Authorization* (204 = aceptado), en dos modos: *keep-alive* (una sesión TCP+TLS reutilizada) y *1 conexión por mensaje*. **Justificación:** IoT Hub no ofrece CoAP; MQTT sobre WebSockets sigue siendo el mismo paradigma que MQTT y no aporta una comparación nueva. HTTPS es nativo en IoT Hub, es el único de los tres sin sesión persistente (petición/respuesta), usa el puerto 443 que casi ninguna red bloquea y lo soporta cualquier microcontrolador con TLS; permite medir el costo de «no tener conexión».

Entorno (importante): la VM `laboratorio3caro` fue eliminada el 2026-10-02 desde una cuenta institucional (Activity Log) y la suscripción «Azure for Students» quedó *Disabled* por crédito agotado, por lo que este laboratorio **no corrió en la VM**: los clientes se ejecutaron en un contenedor Docker Linux (`python:3.12-slim + tcpdump`) en el PC del estudiante, red residencial en Colombia. Pese a ello, IoT Central siguió aceptando telemetría de dispositivos (no se pudieron crear recursos nuevos de Azure, solo usar la app existente). El Lab 3 y sus datos quedaron intactos.

2. Evidencia de funcionamiento

Cada protocolo envió **30/30 mensajes confirmados** (accepted / PUBACK / HTTP 204). Se cruzaron programáticamente los payloads enviados contra *Datos sin procesar* de IoT Central: **AMQP 30/30 y MQTT 30/30 idénticos**; HTTPS 49/49 de las filas visibles (la vista solo lista los últimos ~49 eventos; se enviaron 60). Las tres variables aparecen mapeadas a la plantilla (columnas Humedad relativa, Iluminancia, Temperatura). Detalle en `evidencias/iot-central-verificacion.md`. Bytes medidos en la interfaz del contenedor con `tcpdump` (incluye cabeceras Ethernet/IP/TCP, TLS y ACKs; costo marginal = (30 mensajes – 1 mensaje)/29).

3. Comparación MQTT · AMQP · HTTPS

Criterio	MQTT 3.1.1 (8883)	AMQP 1.0 (5671)	HTTPS (443)
Overhead por mensaje (medido)	417 B en cable (7,5× el payload), 3,9 tramas	474 B (8,5×), 4,0 tramas	keep-alive: 1 069 B (19×; el SAS y las cabeceras HTTP viajan en cada POST). 1 conexión/msj: 7 742 B (138×), 24 tramas
Costo de conexión (medido, 1 mensaje, sin DPS)	7 815 B; conexión+CONNACK 0,46–0,53 s	9 915 B (+27 %: SASL + open/begin/attach); conexión+SASL 0,87–0,91 s, enlace 0,1 s	7 509 B (casi todo TLS); 1.er POST con sesión nueva 557 ms
Latencia de confirmación (medida, 30 msj, sin el 1.º)	media 212,5 ms · mediana 198 · p95 242	media 233 ms · mediana 213 · p95 312	keep-alive: 214 · 208 · 245 ms. 1 conexión/msj: 633 · 619 · 818 ms (≈3× por el handshake TCP+TLS)
Confiabilidad	QoS 0/1 (IoT Hub no soporta QoS 2); PUBACK por mensaje; sesión TCP persistente. Medido: 30/30.	La más rica: disposiciones accepted/rejected/released/modified, modos de liquidación y control de flujo por créditos; varios enlaces por conexión. Medido: 30/30.	Un 204/4xx/5xx por petición; sin orden ni sesión: el reintento y la idempotencia son de la aplicación. Medido: 30/30 (60 con ambos modos).

Microcontrolador / Wokwi (documentado, no medido en este lab)	El mejor soporte: PubSubClient + WiFiClientSecure en ESP32; huella pequeña. En el Parcial 1 el DPS desde Wokwi falló (<code>putCode=-1</code> , red/TLS del simulador); limitación de Wokwi, no del protocolo.	Sin biblioteca ligera y mantenida para Arduino/ESP32 e IoT Hub; el Azure C SDK sí lo soporta, pero es pesado y de portar. No probado en Wokwi.	HTTPClient + WiFiClientSecure nativos y simples, pero cada petición sin keep-alive repite el handshake TLS (costo de CPU/energía en MCU; no medido).
Depuración (medida: clave incorrecta y token vencido)	CONNACK «Not authorized» en ~0,43 s; no distingue causa. Wireshark ve MQTT tras descifrar TLS.	Opaca: el hub cierra el socket durante SASL y el cliente ve «SSL ... unexpected eof» (framing-error); sin código ni mensaje. La más difícil de depurar.	La mejor: HTTP 401 con <code>errorCode 401002</code> , mensaje («invalid signature...») y <code>trackingId</code> ; se prueba con <code>curl</code> . El token vencido solo da «Unauthorized access».
Firewalls / proxies (documentado)	8883 saliente; variante MQTT/WebSockets por 443.	5671 saliente; variante AMQP/WebSockets por 443.	Solo 443 y proxies HTTP(S) estándar: el que más redes atraviesa.
Caso de uso ideal	Telemetría frecuente de dispositivos restringidos, batería/ ancho de banda limitados, comandos nube→dispositivo.	Integración entre servicios (Event Hubs, Service Bus, endpoint compatible con Event Hubs de IoT Hub), pasarelas que multiplexan muchos dispositivos, garantías fuertes de entrega.	Dispositivos que reportan de forma esporádica o desde redes muy restringidas, y pruebas rápidas con <code>curl</code> ; mal candidato para alta frecuencia sin keep-alive.

Lectura de los números: la latencia de ~190–200 ms es casi un RTT hasta el hub, por eso MQTT, AMQP y HTTPS keep-alive no se distinguen en latencia (diferencias dentro del ruido de una sola corrida); la diferencia real está en el overhead por mensaje, en el costo de (re)conectar y en la depurabilidad. **Límites:** una corrida de 30 mensajes por protocolo, una sola red residencial, sin variantes WebSocket ni pruebas en hardware/Wokwi; los cuatro valores «documentados» no deben leerse como medidos.

4. Recomendaciones para la plataforma

- **Protocolo dispositivo → IoT Central: MQTT.** Menor overhead medido (417 B/msj), una conexión persistente, mejor ack medio, mejor soporte en microcontroladores y es el que ya usan los demás nodos (Lab 3 y Parcial 1). Usar MQTT/WebSockets por 443 donde 8883 esté bloqueado.
- **Entre servicios de Azure: AMQP 1.0.** Es el protocolo nativo de Event Hubs/Service Bus y del endpoint de eventos de IoT Hub, con confirmación y control de flujo por enlace; su mayor costo de conexión (+27 %) se amortiza porque en servicios la conexión dura horas y se multiplexan enlaces. No es una buena opción en el dispositivo: handshake más pesado, sin soporte ligero en ESP32 y errores opacos.
- **Los dos protocolos que se conservan: MQTT (dispositivos) y AMQP (backend y pasarelas).** Se descarta HTTPS como protocolo de dispositivo: sin keep-alive cuesta $\approx 138\times$ el payload y $\approx 3\times$ la latencia, y con keep-alive aún cuesta $2,6\times$ el overhead de MQTT (el token SAS va en cada petición); su ventaja de atravesar firewalls la cubren las variantes WebSocket de MQTT/AMQP en 443. HTTPS se mantiene solo como herramienta de diagnóstico (`curl`) por sus mensajes de error claros.
- **Operación:** registrar siempre el `trackingId` y la causa del fallo de autenticación; en AMQP/MQTT, instrumentar el cliente para exponer el código de la desconexión, dado que el hub no lo detalla. Renovar el SAS antes de que venza (3 600 s en este lab).